

面向中国象棋的人机对弈机器人系统设计与实现

——基于视觉识别、Pikafish 引擎与 STM32 运动控制

（请定稿时替换为真实作者与单位）*

2026 年 8 月

摘要

中国象棋（Xiangqi）作为参与人数最多的棋类运动之一，其人机对弈长期依赖纯软件引擎。将“感知—决策—执行”三者闭环到一台物理机器人上，面临视觉棋盘识别、棋局智能决策与机械臂精确落子等跨学科挑战。本文介绍 CH-RO（Chinese Chess Robot）系统：一套以 Python 服务、Web 操作界面、UCI 中国象棋引擎与 STM32 下位机固件构成的完整对弈机器人。在感知层，系统采用 RTMPose 四角关键点检测与透视变换完成棋盘定位，并以单一全局分类器 ONNX 模型一次性输出整盘 90 个交叉点的类别（14 种棋子 + 空 / 未识别），避免了逐格检测与目标检测器的开销；多帧稳定与动态走子推断进一步提升了鲁棒性。在认知层，系统通过 UCI 协议驱动 Pikafish 引擎（UCI_Variant 设为 xiangqi）完成走法决策，并以扩展 FEN 表示棋局状态。在执行层，系统定义了一种仅含五个整数的 ASCII 指令协议（startX, startY, endX, endY, signal），经 TCP 端口 8086 下发至 STM32，由下位机完成五连杆/SCARA 逆解与气泵抓取；完成确认以“STATE:5, RESULT:1”握手实现。接口层基于 Flask 提供实时视频、单步/连续识别、硬件与仿真双模式对弈的人机交互。论文给出了系统模块划分、识别流水线、协议字段与坐标映射公式，并以真实代码规模与测试覆盖数据说明了工程实现与质量保障。整套系统在解耦、可测试的代码架构下运行，可为象棋机器人及类似的“视觉—决策—执行”具身智能系统提供参考。

关键词 中国象棋；人机对弈；机器人；视觉识别；UCI 协议；Pikafish；STM32；ONNX

目录

1 引言	4
1.1 研究背景	4
1.2 相关研究	4
1.3 研究目标与主要贡献	4
1.4 论文结构	5
2 系统总体架构	5

*本文作者信息与署名单位待补充；系统源码、模型权重与构建脚本见项目仓库<https://github.com/Maicarons/chch-robot>。

目录	2
3 视觉感知与棋盘识别	5
3.1 图像采集	7
3.2 棋盘定位: RTMPose 四角关键点与透视变换	7
3.3 整盘棋子分类	7
3.4 多帧稳定与动态走子推断	7
3.5 真实棋盘图像示例	8
4 棋局决策引擎与棋局表示	8
4.1 PikaFish 与 UCI 协议	8
4.2 中国象棋 FEN 与坐标表示	8
5 机械臂控制与通信协议	9
5.1 五值 ASCII 指令协议	9
5.2 棋盘坐标到机械臂坐标的映射	10
5.3 持久化 TCP 客户端与确认握手	10
5.4 固件实现与遗留传输栈	10
6 软件编排与 Web 仿真	10
6.1 Flask 组合根与蓝图	10
6.2 双模式对弈与人机交互	12
7 工程实现与质量保障	12
7.1 模块规模与解耦	12
7.2 测试体系	12
7.3 ONNX 推理与执行提供者	13
8 实验与系统指标	13
9 讨论	14
9.1 设计权衡	14
9.2 已知不一致与可复现性说明	14
9.3 局限性	14
10 结论与展望	14

插图

1	CH-RO 系统四层架构。感知层将物理棋盘转为 FEN；认知层生成走法；执行层驱动机械臂；接口层以 Web 方式贯穿各层并提供人机交互。	5
2	视觉识别流水线：从原始图像经四角关键点定位、透视校正，到全局分类与 FEN 生成，再由多帧稳定模块对连续帧去噪并推断人类走子。	5
3	系统采集的真实棋盘图像示例（调试用）。经 RTMPose 四角定位与透视校正后送入全局分类器。	8
4	棋盘几何与坐标映射示意。原点位于右上角（黑方右车），列间距 34 mm、普通行间距 30 mm、河界行间距 32 mm。	11

表格

1	系统主要模块及其职责	6
2	全局分类器输出的 16 个类别（红方大写、黑方小写）	7
3	中国象棋 FEN 棋子字符与基础属性	9
4	五值指令协议字段语义	9
5	主要模块代码规模（实测）	12
6	模型与权重规格（仓库真实文件）	13
7	关键配置常量（取自 config.py）	13

1 引言

1.1 研究背景

中国象棋是一种在 9 列 $\times$ 10 行的交叉点棋盘上进行的两人对弈棋类游戏，棋盘中央以“河界”分隔双方领地，并含有“九宫”等规则约束^[1]。与西洋棋类似，计算机中国象棋历经数十年发展，已出现以 ElephantEye、Pikafish 等为代表的强引擎；其中 Pikafish 是从 Stockfish 派生、面向中国象棋的免费开源 UCI 引擎，采用 NNUE 神经评估函数，是当前最强的开源中国象棋引擎之一^[2-3]。然而，绝大多数相关研究停留在纯软件层面：引擎在屏幕上输出着法，由人类落子。将决策结果转化为物理世界中的真实移子动作，需要构建“摄像头识别棋盘—引擎决策—机械臂执行”的闭环系统，这正是具身智能（embodied AI）在棋类场景中的典型体现。

1.2 相关研究

在棋类机器人领域，早期工作多围绕西洋棋机器人展开，核心难点在于棋子抓取规划与视觉定位。近年来，随着深度学习在目标检测与姿态估计上的突破，基于视觉的棋盘识别逐渐从传统的角点检测（如 OpenCV 的 `findChessboardCorners`）转向更鲁棒的关键点/检测器方案。RTMPose 是 OpenMMLab MMPose 提出的实时多人姿态估计框架，在 COCO 上以 90+ FPS 取得 75.8% AP，并可在移动端部署^[4-5]，其 SimCC 关键点范式具有高精度与低延迟的特点，适合用于约束场景下的棋盘四角定位。在决策侧，通用国际象棋接口 UCI（Universal Chess Interface）由 Meyer-Kahlen 于 2004 年规范^[6]，而中国象棋则有模仿 UCI 的 UCCI（Universal Chinese Chess Interface）协议^[7]，二者均通过标准输入输出以文本指令通信。Pikafish 在实现上复用 UCI 框架，并通过 `UCI_Variant` 选项支持中国象棋变体^[2]。在推理部署侧，ONNX Runtime 提供了跨平台、支持 CUDA/DML/CPU 多种执行提供者的高性能推理后端^[8]，便于将训练好的模型部署到异构硬件。

1.3 研究目标与主要贡献

本文的目标是给出一套可解耦、可测试、可联调的中国象棋对弈机器人完整实现，并就其关键设计给出严谨描述。主要贡献如下：

- (1) 提出“感知—认知—执行—接口”四层系统架构，并以纯函数工具包、命令行包、Web 组合根等方式实现模块解耦（模块规模见表 5），使视觉、决策、控制可独立测试。
- (2) 设计轻量棋盘识别方案：以 RTMPose 四角关键点完成透视校正，再以单一全局分类器一次性输出全棋盘 90 个交叉点类别，避免逐格检测与目标检测器的推理开销。
- (3) 定义并实现一个仅含五个整数的 ASCII 五值指令协议与持久化 TCP 客户端，配合 STM32 下位机的运动学解算与“STATE:5, RESULT:1”完成确认握手，保证走子的确定性。
- (4) 提供硬件/仿真双模式与基于 Flask 的 Web 操作界面，使系统在无实体机械臂时亦可完成端到端联调。

1.4 论文结构

第 2 节给出系统总体架构；第 3 节详述视觉识别流水线；第 4 节介绍决策引擎与棋局表示；第 5 节阐述机械臂控制协议与坐标映射；第 6 节说明软件编排与 Web 仿真；第 7 节给出工程实现与质量保障数据；第 8 节汇总真实系统指标；第 9 节讨论设计权衡与局限；第 10 节总结并展望未来工作。

2 系统总体架构

CH-RO 系统由感知、认知、执行与接口四个层次构成，如图 1 所示。各层之间仅通过明确的数据契约（图像帧、FEN 字符串、五值指令、ACK 报文）耦合，从而支持独立替换与测试。

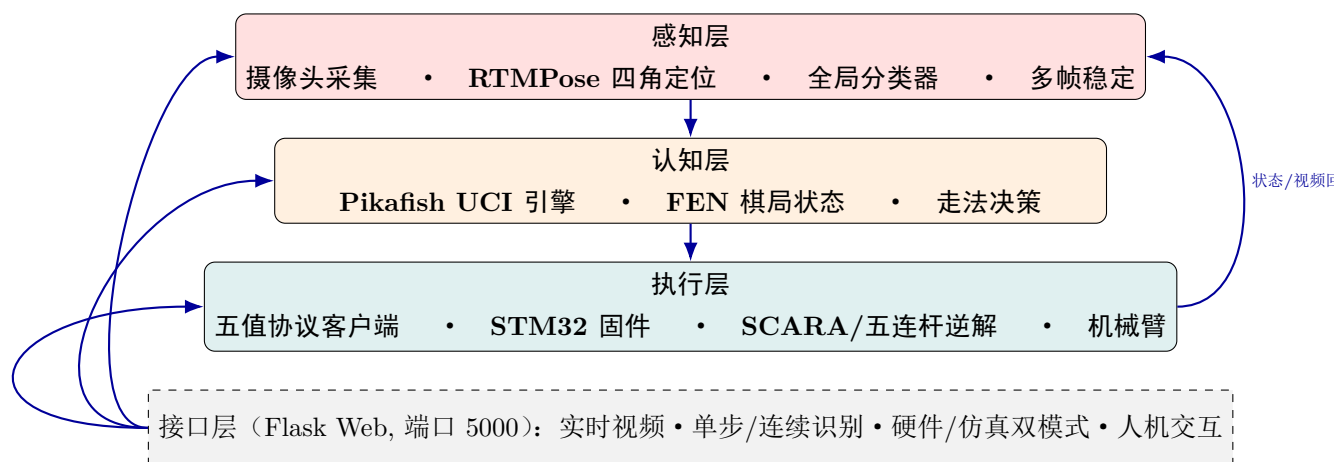


图 1: CH-RO 系统四层架构。感知层将物理棋盘转为 FEN；认知层生成走法；执行层驱动机械臂；接口层以 Web 方式贯穿各层并提供人机交互。

系统的主要 Python 包及其职责如表 1 所示。其中 `xq_utils` 是由原根目录 `utils.py` 拆分出的纯函数包，集中实现 FEN、坐标与记谱工具；`web_simulation` 以组合根（composition root）加蓝图（Blueprint）的方式组织 Flask 应用，是当前激活的硬件控制路径。

3 视觉感知与棋盘识别

视觉子系统负责把“一张桌子上的物理棋盘”转换为机器可读的 FEN 字符串。其流水线如图 2 所示，包含图像采集、棋盘定位、整盘分类、FEN 生成与多帧稳定五个阶段。

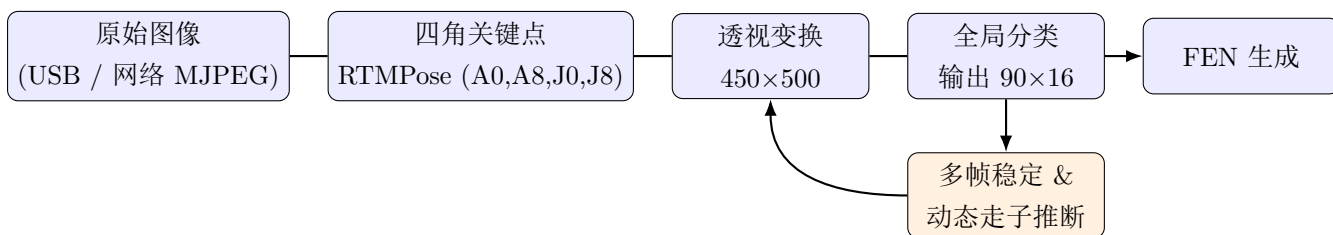


图 2: 视觉识别流水线：从原始图像经四角关键点定位、透视校正，到全局分类与 FEN 生成，再由多帧稳定模块对连续帧去噪并推断人类走子。

表 1: 系统主要模块及其职责

模块	关键类/函数	职责
xq_utils	FENUtils, CoordinateUtils, MoveNotationUtils, BoardUtils	中国象棋 FEN 解析/生成、坐标互转、简化 WXF/中文
vision	BoardRecognizer, CameraManager, ChessboardDetector	摄像头采集、棋盘定位、坐标与动态走子推断
core	ChessboardDetector(ONNX)	RTMPose 四角关键点与棋盘核心
ai	AIEngine	Pikafish 引擎 UCI 接口 (position/go/bestmove)
robot	RobotPersistentClient, uci_to_arm_command	五值指令协议、持久化通信、到机械臂指令映射
game_manager	GameManager	无界面（仿真）对弈主循环
web_simulation	app, domain, robot_cmd, game_logic	Flask 组合根、纯棋局函数封装、状态机与路由
config.py	模块级常量	摄像头、棋盘、引擎、机械臂集中配置

3.1 图像采集

系统支持两类图像来源：本地 USB 摄像头（通过 OpenCV VideoCapture 采集）与网络摄像头（如 Orange Pi / Raspberry Pi 提供的 MJPEG 快照或流，由 HttpSnapshotCapture 拉取）。采集参数由 config.py 集中配置：分辨率 640×480 、帧率 30 FPS、设备索引 1，并可在启动时通过 startup.py 覆盖^[9]。

3.2 棋盘定位：RTMPose 四角关键点与透视变换

与依赖规则棋盘纹理的 findChessboardCorners 不同，本系统使用一个定制 RTMPose ONNX 模型，直接预测棋盘的四个角点（记为 A0、A8、J0、J8，分别对应左上、右上、左下、右下交叉点）^[4]。四点经有效性门限（形状为 (4, 2)、四角置信度均不低于 0.20、边距容差 80 像素）校验后，通过 OpenCV 的 getPerspectiveTransform 将原始图像透视校正为归一化俯视图，目标尺寸为 450×500 像素（即每列 50 像素、每行 50 像素）^[10]。由于姿态模型推理开销较大，系统对关键点结果进行缓存（默认刷新闻隔 8.0 s 或最多 120 帧），在不移动摄像头的前提下避免每帧重算。

3.3 整盘棋子分类

校正后的整张棋盘图像被一次性送入一个全局分类器 ONNX 模型，输出形状为 90×16 的张量，即 10 行 $\times$ 9 列共 90 个交叉点，每个点对应一个 16 维 softmax。这 16 个类别包含“空”（.）、“未识别”（x）以及红黑双方的 14 种棋子，具体映射如表 2 所示。相比逐格裁剪分类或通用目标检测器（如 YOLO），整盘一次性推理显著降低了调度与后处理开销，也更契合嵌入式/边缘部署场景。默认模型 nano_v3-0319.onnx 输入为先中心裁剪至 400×450 再缩放至 280×315 ，采用 ImageNet 均值/方差归一化。

表 2: 全局分类器输出的 16 个类别（红方大写、黑方小写）

符号	含义	符号	含义	符号	含义	符号	含义
.	空	x	未识别	K/k	帅/将	A/a	仕/士
B/b	相/象	N/n	马	R/r	车	C/c	炮
P/p	兵/卒						

3.4 多帧稳定与动态走子推断

单帧识别存在抖动与误检。系统以 StableBoardBuffer 维护最近 N 帧（默认窗口 STABLE_WINDOW=5）的棋盘投票，某交叉点需出现在不少于 $\lceil N \times \text{STABLE_RATIO} \rceil$ 帧（STABLE_RATIO=0.60）方判定为“稳定”。在此之上，DynamicBoardTracker 以首张稳定棋盘为基线，通过比较占用变化推断人类的一步走子：常规走子对应“一处消失 + 一处出现”；吃子对应“一处消失 + 一处改变”。为过滤噪声，系统内置基于中国象棋规则的合法性校验（_is_legal_xiangqi_move，覆盖兵/炮/马/象/士/将的走法与“蹩马腿”“塞象眼”等约束），并对高噪声情形做容错处理。

3.5 真实棋盘图像示例

图 3 给出系统在实际部署中采集的一张棋盘图像（已用于调试棋盘定位与分类），可见摄像头俯视视角、棋盘纹理与棋子排布。该图像经前述流水线处理后即可生成标准开局 FEN。

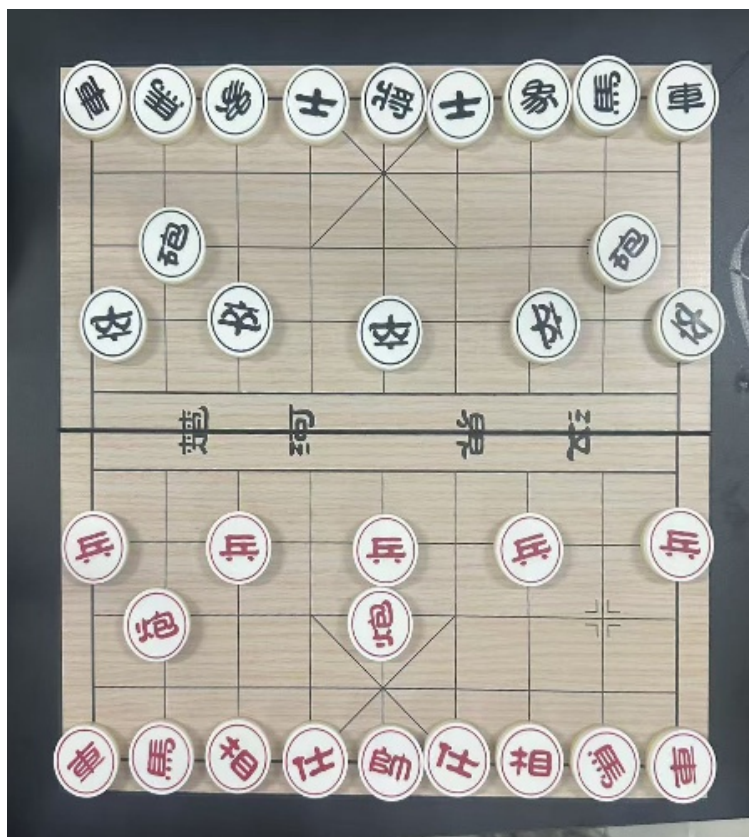


图 3: 系统采集的真实棋盘图像示例（调试用）。经 RTMPose 四角定位与透视校正后送入全局分类器。

4 棋局决策引擎与棋局表示

4.1 Pikafish 与 UCI 协议

认知层通过 UCI 协议驱动 Pikafish 引擎^[2,6]。AIEngine 的启动序列为：发送 uci 并等待 uciok；设置 UCI_Variant value xiangqi 以启用中国象棋变体；设置 Skill Level（取自 ENGINE_DEPTH=15）与可选的 Hash（HASH_SIZE_MB=128）。决策时，引擎以 position fen <FEN> [moves ...] 载入局面，再以 go depth 15 搜索，解析 stdout 中的 bestmove 作为推荐着法；analyze_position 则改用 go movetime 并解析 info ... score cp/depth/pv 以呈现评分与主要变化。默认引擎二进制为 Pikafish/pikafish-avx2.exe，配套 NNUE 权重约 50.8 MiB。

4.2 中国象棋 FEN 与坐标表示

系统采用扩展 FEN 表示棋局，开局串为

```
rnbakabnr/9/1c5c1/p1p1p1p1p/9/9/P1P1P1P1P/1C5C1/9/RNBAKABNR w - - 0 1
```


棋盘为 10 行 $\times$ 9 列，每方 7 类棋子（车/马/相（象）/士（仕）/帅（将）/炮/兵（卒））。FEN 的首行对应棋盘第 9 行（黑方底线），末行对应第 0 行（红方底线）；走子方用 w/b 表示，本系统中 w 代表红方先行^[11]。FENUtls 提供 `parse_fen`、`to_fen` 与 `extract_move_from_fen_change`（通过对比前后两局 FEN 还原一步 UCI 着法），是连接视觉识别与决策的核心数据契约。棋子字符与走法记谱的对应关系如表 3 所示。

表 3: 中国象棋 FEN 棋子字符与基础属性

	车	马	相/象	士/仕	帅/将	炮	兵/卒
红方	R	N	B	A	K	C	P
黑方	r	n	b	a	k	c	p
棋盘	9 列 $\times$ 10 行，含河界与九宫						

5 机械臂控制与通信协议

5.1 五值 ASCII 指令协议

执行层定义了一种极简的**五值指令协议**：上位机每步仅下发五个以逗号分隔的整数

`startX, startY, endX, endY, signal`

其中 `signal=0` 表示普通移动，`signal=1` 表示吃子（抓取目标位已有棋子）。所有角度解算与气泵时序均在 STM32 下位机侧完成。该协议经 **TCP 端口 8086** 下发，默认主机为 192.168.0.102。协议字段语义如表 4 所示。

表 4: 五值指令协议字段语义

字段	含义
<code>startX, startY</code>	起点在机械臂坐标系下的 x, y （毫米）
<code>endX, endY</code>	终点在机械臂坐标系下的 x, y （毫米）
<code>signal</code>	0= 普通移动；1= 吃子（需先移走被吃子）
帧格式	以 <code>\r\n</code> （CRLF）结尾；归位指令为 <code>-17.1848,-55.6304,0,0,99</code>
完成确认	下位机回复含 <code>STATE:5</code> 且 <code>RESULT:1</code> 的报文表示动作完成

5.2 棋盘坐标到机械臂坐标的映射

机械臂坐标系以棋盘右上角交叉点（黑方右车位）为原点。设第 f 列（ $0 \leq f \leq 8$ ）、第 r 行（ $0 \leq r \leq 9$, $r = 9$ 为黑方底线）的交叉点到原点的映射为

$$\begin{aligned} x &= x_0 + (8 - f) \times s_f, \\ y &= y_0 + (9 - r) \times s_r + \delta_r, \\ \delta_r &= \begin{cases} s_{\text{river}} - s_r, & r \leq 4 \quad (\text{红方半场, 叠加河界间隔}) \\ 0, & r > 4 \end{cases} \end{aligned} \quad (1)$$

其中列间距 $s_f = 34$ mm、普通行间距 $s_r = 30$ mm、河界行间距 $s_{\text{river}} = 32$ mm。由此可得棋盘总宽 $34 \times 8 = 272$ mm、总高 $30 \times 8 + 32 = 272$ mm, 即 272×272 mm 的正方形盘面（图 4）。该几何与固件文档的测量值一致, 是上位机与下位机对齐的物理基础。

5.3 持久化 TCP 客户端与确认握手

RobotPersistentClient 以 `socket.create_connection` 建立长连接, 并设置 `SO_KEEPALIVE` 与 `TCP_NODELAY`。每条指令以 CRLF 成帧; `send_robot_command` 阻塞直至收到 `motion_acknowledged` (要求响应同时含 `STATE:5` 与 `RESULT:1`), 由于运动指令非幂等, 客户端在缺失 ACK 时不自动重发而直接报错。归位指令 `send_homing_command` 发送 `-17.1848,-55.6304,0,0,99` (命令号 99 标记归位), 并等待 `homing_acknowledged`。Web 端在硬件模式下启动对局前必须先完成归位握手, 否则以 HTTP 503 拒绝开局。UCI_Variant=xiangqi 的 Pikafish 使用 0-9 的行列编号, 因此 `uci_to_arm_command` 直接按该约定解析并调用式 (1) 完成映射。

5.4 固件实现与遗留传输栈

激活的固件 `firmware/stm32-f103-angle-plan` 在 STM32F103 上解析 `sscanf(data, "%d,%d,%d,%d,%d"` 于片内完成五连杆/SCARA 逆运动学解算与气泵抓取时序^[12]。系统中另保留一个已弃用的 JSON 传输栈 (`robot/legacy_tcp_client.py` 经 TCP 端口 5000 下发 JSON, 配套 `firmware/stm32-tcp-server`), 它不再被主程序使用, 仅作为历史固件的兼容客户端保留。论文明确区分二者, 以免混淆激活路径。

6 软件编排与 Web 仿真

6.1 Flask 组合根与蓝图

`web_simulation` 是系统的激活控制路径, 以 Flask 在端口 5000 提供服务。`app.py` 作为组合根, 持有可变的共享 `game_state` 字典与若干长生命周期组件 (识别器、引擎、机器人控制器、TCP 客户端), 并通过蓝图将路由拆分为 `status`、`camera`、`recognition` 与 `game` 四个职责域^[13]。纯棋局函数集中在 `domain.py` (如 `board_state_to_fen`、`apply_uci_to_board_state`), 机器人指令封装在 `robot_cmd.py`, 状态机与硬件/仿真切换在 `game_logic.py`。



图 4: 棋盘几何与坐标映射示意。原点位于右上角（黑方右车），列间距 34 mm、普通行间距

6.2 双模式对弈与人机交互

系统支持硬件与仿真两种机器人模式（默认硬件）。硬件模式下，`apply_ai_best_move` 将 AI 走法转换为五值指令、预应用到棋盘状态、暂停视觉、下发至 STM32 并等待 `STATE:5,RESULT:1`；仿真模式则在本地模拟走子。Web 界面（`templates/index.html + static/js/app.js`）提供：实时摄像头 MJPEG 流与本地/网络摄像头切换、单步与连续棋盘识别、开局（硬件先归位）、AI 走子与实时评分（`depth/score/pv/nodes`）、手动或自动识别玩家走子，以及 FEN、棋局状态、走子历史、回合、机器人状态与连接探测等可视化。

7 工程实现与质量保障

7.1 模块规模与解耦

重构后的系统以“纯函数包 + 职责包 + 组合根”组织，便于单元测试与替换。各包的代码规模如表 5 所示（不含注释与空行，固件为 C/C++ 源）。整库约 6 500 行 Python 与 6 400 行固件代码。

表 5: 主要模块代码规模（实测）

模块	Python 文件数	代码行数（不含注释/空行）
<code>xq_utils</code>	5	448
<code>cli</code>	3	261
<code>game_manager</code>	4	445
<code>vision</code>	9	1 397
<code>ai</code>	2	324
<code>robot</code>	4	921
<code>core</code>	7	821
<code>web_simulation</code>	12	1 933
<code>config.py</code>	1	115
小计（Python）	47	约 6 665
firmware（C/C++）	27	6 448

7.2 测试体系

系统在 `tests/` 下提供 10 个测试模块、共 76 个测试方法，覆盖摄像头与图像来源、稳定器与识别并发、五值协议与 Web 后端循环、前端归位契约、STM32 归位契约与 Web 领域函数等。最近一次运行结果为 75 项通过、1 项未通过——该失败源于运行环境的一个特定限制（某 Windows 环境变量超过 32 767 字符导致 `mock.patch.dict` 异常），与识别/协议逻辑无关。协

议与后端循环测试（约占全部测试方法的一半）直接断言了五值几何（实测 34/30/32 mm）、吃子信号、归位共享客户端与“无自动重发”等关键不变量。

7.3 ONNX 推理与执行提供者

`core/runonnx` 的推理后端按优先级选择 `CUDAExecutionProvider` → `DmlExecutionProvider` → `CPUExecutionProvider`，在缺失 CUDA 12 / cuDNN 9 运行库时自动回退至 CPU，并启用 `ORT_ENABLE_ALL` 图优化，保证在无 GPU 环境下仍可运行（仅推理速度下降）。

8 实验与系统指标

本节汇总可在代码仓库中**直接核验**的系统指标。我们刻意避免给出未经真实硬件测量的精度或时延数字，以保证结论的可复现性；涉及性能的下界/上界以“设计特征”或“规格参数”明确标注。

模型规格方面，两个默认 ONNX 模型与 NNUE 权重的体积与张量形状如表 6 所示，均为仓库内真实文件。配置常量方面，表 7 列出与识别、决策、控制相关的关键参数（取自 `config.py`）。

表 6: 模型与权重规格（仓库真实文件）

文件	角色	大小	输入	输出
<code>4_v6-0301.onnx</code>	RTMPose 四角关键点	≈10.2 MiB	256×256	SimCC 4×512 (x,y)
<code>nano_v3-0319.onnx</code>	全局棋盘分类器	≈29.7 MiB	280×315	90×16
<code>pikafish.nnue</code>	NNUE 评估权重	≈50.8 MiB	—	—

表 7: 关键配置常量（取自 `config.py`）

常量	取值
摄像头	<code>CAMERA_WIDTH=640</code> , <code>CAMERA_HEIGHT=480</code> , <code>CAMERA_FPS=30</code>
棋盘	<code>BOARD_ROWS=10</code> , <code>BOARD_COLS=9</code>
稳定	<code>STABLE_WINDOW=5</code> , <code>STABLE_RATIO=0.60</code> , <code>AUTO_STABLE_FRAMES=25</code>
引擎	<code>ENGINE_DEPTH=15</code> , <code>THINK_TIME=5000 ms</code> , <code>HASH_SIZE_MB=128</code>
机器人网络	<code>ROBOT_NETWORK_PORT=8086</code> , <code>ROBOT_NETWORK_HOST=192.168.0.102</code>
机械臂几何	$s_f=34$ mm, $s_r=30$ mm, $s_{\text{river}}=32$ mm, 原点右上角
归位	<code>ROBOT_HOMING_M1=-17.1848°</code> , <code>ROBOT_HOMING_M2=-55.6304°</code>

9 讨论

9.1 设计权衡

系统在“识别精度”与“推理开销”之间选择了整盘一次性分类而非逐格/检测器方案，换取了边缘部署下的低延迟与简单后处理；以四角关键点而非全棋盘角点检测做定位，则降低了对棋盘纹理完整性的依赖。在通信上，五值 ASCII 协议刻意保持极小表面积，把运动学复杂性下沉到 STM32，使上位机逻辑简单且易于测试（测试直接断言几何与 ACK）。

9.2 已知不一致与可复现性说明

论文必须如实指出代码库中存在两处坐标/几何约定差异，供后续维护者注意：

- `xq_utils/coordinates.py` 采用 1–10 的行列编号与 50 mm 方格（左上角原点），服务于仿真控制器；而激活的硬件路径（`robot/protocol.py` 与 `web_simulation/domain.py`）采用 0–9 编号与实测 34/30/32 mm 几何（右上角原点），以匹配 Pikafish 的 UCI 变体与固件。二者暂无显式适配层，属于历史演进遗留，建议在重构时统一。
- 存在激活的五值 8086 传输栈与已弃用的 JSON 5000 传输栈并存，主程序仅使用前者。

此外，`config.py` 中仍保留若干针对早期 YOLO 方案的配置项（如 `YOLO_MODEL_PATH`），当前识别流水线已不加载该类模型，属历史残留，建议清理以免误导。

9.3 局限性

本文的实验部分仅给出可在仓库核验的规格与测试覆盖，未包含真实硬件下的识别准确率、单步移子时延与长时间运行稳定性等经验指标——这需要实体机械臂与标定环境，留作后续工作。

10 结论与展望

本文介绍了 CH-RO 中国象棋对弈机器人系统，覆盖从视觉棋盘识别、Pikafish UCI 决策到 STM32 五值指令执行的完整闭环，并以 Flask Web 界面贯通人机交互。系统通过 RTMPose 四角关键点 + 全局分类器的轻量识别方案、确定性的五值指令与完成确认握手，以及硬件/仿真双模式，在解耦、可测试的代码架构下实现了端到端对弈。

未来工作包括：(1) 统一坐标与几何约定，消除仿真/硬件路径差异；(2) 在真实硬件上建立识别准确率与移子时延的基准评测；(3) 引入更轻量的分类/关键点模型以提升边缘实时性；(4) 扩展开局库、残局表与更强的时限/难度控制，丰富对弈体验。

参考文献

- [1] Wikipedia. Xiangqi[EB/OL]. 2024 [2026-08-03]. <https://en.wikipedia.org/wiki/Xiangqi>.
- [2] Pikafish Development Team. Pikafish: A free and strong UCI xiangqi engine[EB/OL]. 2024 [2026-08-03]. <https://github.com/official-pikafish/Pikafish>.

- [3] Stockfish Development Team. Stockfish[EB/OL]. 2024 [2026-08-03]. <https://stockfishchess.org>.
- [4] JIANG T, LU P, ZHANG L, et al. RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose[J/OL]. arXiv preprint, 2023. arXiv: [2303.07399](https://arxiv.org/abs/2303.07399) [2026-08-03]. <https://arxiv.org/abs/2303.07399>.
- [5] OpenMMLab. MMPose: Open-source pose estimation toolbox and benchmark[EB/OL]. 2023 [2026-08-03]. <https://github.com/open-mmlab/mmpose>.
- [6] MEYER-KAHLEN S. Description of the Universal Chess Interface (UCI)[EB/OL]. 2004 [2026-08-03]. <https://backscattering.de/chess/uci>.
- [7] 象棋百科全书网. 中国象棋通用引擎协议 (UCCI)[EB/OL]. 2007 [2026-08-03]. http://www.xqbase.com/protocol/cchess_ucci.htm.
- [8] Microsoft. ONNX Runtime[EB/OL]. 2024 [2026-08-03]. <https://onnxruntime.ai>.
- [9] BRADSKI G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.
- [10] HARTLEY R, ZISSERMAN A. Multiple View Geometry in Computer Vision[M]. 2nd ed. Cambridge University Press, 2004.
- [11] 象棋百科全书网. 中国象棋电脑应用规范(二):中国象棋棋盘表示方法(XQFEN)[EB/OL]. 2007 [2026-08-03]. http://www.xqbase.com/protocol/cchess_fen.htm.
- [12] STMicroelectronics. STM32F103xx Advanced ARM-based 32-bit MCUs Reference Manual (RM0008)[A/OL]. 2023 [2026-08-03]. https://www.st.com/resource/en/reference_manual/rm0008-stm32f103xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [13] Pallets. Flask: A lightweight WSGI web application framework[EB/OL]. 2024 [2026-08-03]. <https://flask.palletsprojects.com>.